UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA

JULIANA RODRIGUES

Heaps k-ários e Algoritmo de Dijkstra

Porto Alegre

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Reitora: Prof[a]. Marcia Cristina Bernardes Barbosa
Vice-Reitor: Prof. Pedro de Almeida Costa
Pró-Reitora de Pós-Graduação: Prof[a]. Claudia Wasserman
Diretor do Instituto de Informática: Prof. Luciano Paschoal Gaspary
:
Bibliotecário-chefe do Instituto de Informática: Alexsander Borges Ribeiro

1 RELATÓRIO DO EXPERIMENTO

1.1 Introdução

Esse trabalho descreve a implementação e análise de desempenho do algoritmo de Dijkstra utilizando um Heap k-ário. O relatório apresenta uma análise de complexidades teóricas e valida o algoritmo com teste no grafo da rede rodoviária dos Estados Unidos.

1.2 Detalhes de Implementação

1.2.1 Heap k-ário

1.2.1.1 Busca de vértice

```
std::vector<HeapNode> heap_;
std::vector<int> pos_;
```

- Complexidade: O array *pos_* guarda o índice de cada vértice dentro do Heap. Com isso, o custo de localizar um vértice para atualização é $O(1)$.

1.2.1.2 $sift_{up}$

```
    void sift\textsubscript{up}(int i) {
    while (i > 0 && heap_[parent(i)].distance > heap_[i].distance) {
        int p = parent(i);
        std::swap(heap_[i], heap_[p]);
        pos_[heap_[i].vertex] = i; // Atualiza o mapeamento O(1)
        pos_[heap_[p].vertex] = p;
        i = p;
    }
}
```

- Complexidade: A operação sobre na árvore comparando o nó com seu pai direto. Percorrer a árvora k-ária no pior caso (da folha à raiz) custa $O(log_k n)$.

1.2.1.3 $sift_{down}$

```
    void sift_down(int i) {
        while (true) {
            int min_child = -1;
            // Inspeciona até k filhos para encontrar o menor
            for (int j = 1; j <= k_; ++j) {
                int child_pos = kth_child(i, j);
                if (child_pos < heap_.size()) {
                    if (min_child == -1 ||
                    heap_[child_pos].distance < heap_[min_child].distance) {
                        min_child = child_pos;
                    }
                }
            }
        }
    }
```

- Complexidade: A operação de descida na árvore, no pior caso, exige em cada nível a comparação com os $k-1$ filhos para determinar o menor caminho. Assim, o custo é $O(klog_k n)$

1.2.1.4 Outras operações

As seguintes usam as de complexidade já definidas acima.

- Insert: Insere no final do array e chama $sift_{up}$. Complexidade: $O(log_k n)$
- $\text{Decrease}_{\text{key}}$ (Update): Acessa a posição via *pos_* e chama $sift_{up}$. Complexidade: $O(log_k n)$
- $\text{Extract}_{\text{min}}$ (Delete_min): Remove a raiz (trocando com o último elemento) e chama $sift_{down}$. Complexidade: $O(klog_k n)$

1.2.1.5 Dijkstra

```
while (!pq.is_empty()) {
    int u = pq.extract_min();
    for (const auto& edge : graph_.get_neighbors(u)) {
        int v = edge.to;
        double weight = edge.weight;
```

```
        if (distances_[u] + weight < distances_[v]) {
            distances_[v] = distances_[u] + weight;

            if (!pq.contains(v)) {
                pq.insert(v, distances_[v]);
            } else {
                pq.decrease_key(v, distances_[v]);
            }
        }
    }
}
```

- Complexidade Total: Somando os custos já definidos, a complexidade do algoritmo é $n * klog_k n + m * log_k n$

1.2.2 Representação do Grafo

O grafo foi representado como uma lista de adjacência, que é mais eficiente que uma matriz de adjacência para grafos esparsos.

```
\struct Edge {
    int to;
    double weight;
};
class SparseGraph {
private:
    int num_vertices_;
    std::vector<std::vector<Edge>> adj_;
    // ...
public:
    void add_edge(int u, int v, double weight) {
        adj_[u].push_back({v, weight});
    }
};
```

- Complexidade de espaço: $O(n+m)$. A lista armazena n nodos + m arestas.
- Inserção de aresta: $O(1)$, devido ao uso da função $push_{back}$ da biblioteca *std::vector*.

1.3 Ambiente de Teste

Notebook Dell Pro 14 Plus, equipado com processador Intel Ultra 5 e 16 GB de memória RAM.

1.4 Resultados Experimentais e Análise

Para os gráficos com *box plots* e faixas de variância contínua, é importante considerar que eventuais variâncias muito elevadas podem ser resultado de ruídos de concorrência com o sistema operacional no momento dos testes.

1.4.1 Ajuste do Grau Ideal para o Heap k-ário

A aplicação do algoritmo de Dijkstra exige três elementos fundamentais:

- A implementação do algoritmo.
- Um heap k-ário que armazena as distâncias.
- Uma fila de prioridade, que indica qual será o próximo nodo explorado

A complexidade teórica do algortimo de Dijkstra é $0(n)+n*deletemin+n*insert+m*update$. A complexidade concreta depende de como as operações são implementadas. O grau k do Heap afeta o tempo de execução das operações de *deletemin*, *insert* e *update*.

O primeiro teste busca definiu o grau ideal k do heap experimentalmente. Para isso, foram exploradas as combinações com os graus k = 2,3,4,5,8 e 16:

1. Número de nodos(n) fixado em 100000, número de arestas: m = 2n, m = 10n, m = 30n.
2. Número de nodos(n) fixado em 250000, número de arestas: m = 2n, m = 10n, m = 30n.

A variação do número de arestas é feita para variar a densidade dos grafos em: esparsos (n * 2), médios (n * 10) e densos (n * 30). Cada experimento foi replicado 10 vezes.

O resultado dos experimentos 1. e 2. está nos gráficos das Figuras 1.1 e 1.2. Pode-se observar:

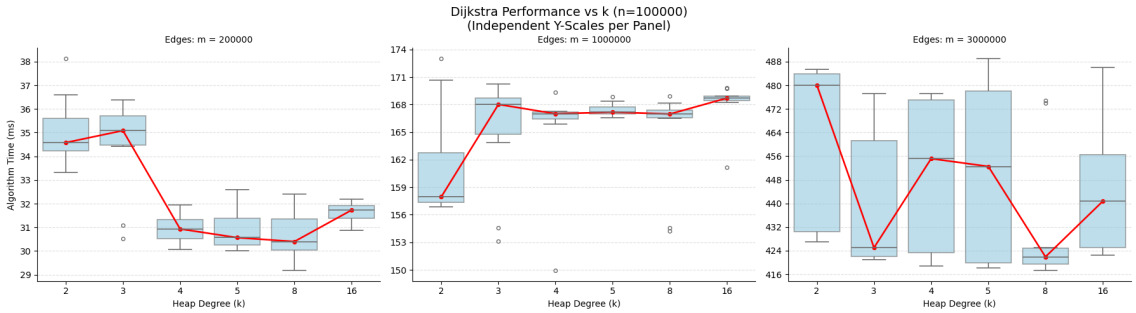

Figura 1.1 – Tempo de execução em relação ao grau do heap k para n = 100.000.

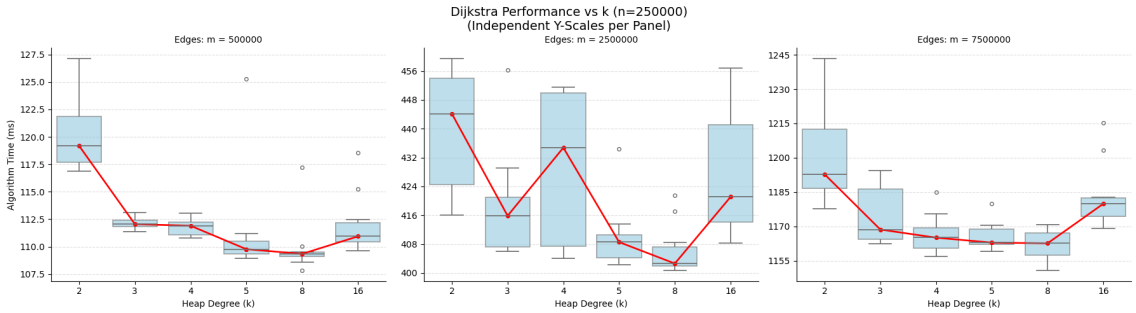

Figura 1.2 – Tempo de execução em relação ao grau do heap k para n = 250.000.

- Heaps com menor k resultam em árvores profundos. Nessas topologias, as operações de subida na árvore (Update/sift_{up}) são mais lentas, e as operações de descida (Insert/$\text{sift}_{\text{down}}$) são mais rápidas. Heaps com maior k são mais largos, e para eles vale contrário.

  Heaps com maior número m de arestas, para o mesmo número de nodos são mais densos e precisam de mais operação de Update. Isso faz com que eles se beneficiem de um maior k. É possível ver no gráfico que na maioria das curvas os pontos tem menor valor no eixo de tempo indo para direita. O comportamento começa a inverter com k=16, em que número aumentado de tempo para operaçãos de Inserts/$\text{sift}_{\text{down}}$ supera a economia de tempo nas operações de Updates/sift_{up}.

- Alguns pontos fogem da tendência citada acima. Nesses pontos, como k = 4 e k = 5 no gráfico da figura 1.1 o tamanho da caixa de variância é maior. Nesse caso, a diferença entre os resultados de cada replicação foi grande, e condições externas ao algoritmo, como outros processos rodando na máquina, podem ter prejudicado o teste.

Assim, foi obtido das simulações o grau ideal para o Heap = 8, especialmente em grafos densos.

1.4.2 Validação da Complexidade do Heap

No Heap k-ário o número de sifts é limitado matematicamente pela altura da árvore ($log_k n$). A fração de *sifts* realizada pelo código não ultrapassa o limite teórico (1.0), como visto na Fig. 1.3. Grafos mais densos demandam mais operações de sift_{up} e $\text{sift}_{\text{down}}$.

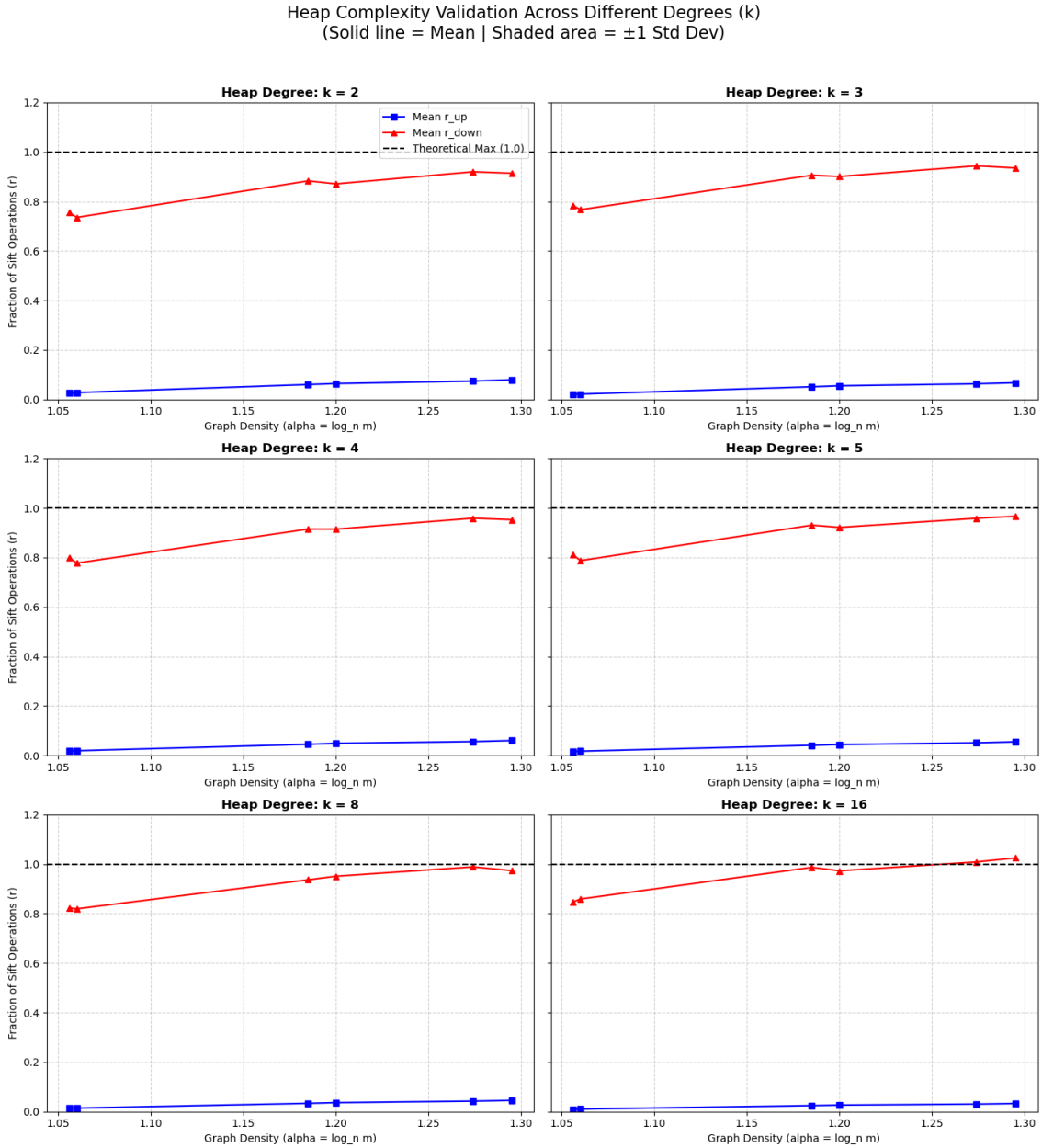

Figura 1.3 – Fração das operações de sift em função da densidade do grafo para diferentes graus do heap (k).

1.4.3 Validação da Complexidade do Algoritmo: Operações de Atualização

Na Fig. 1.4 é possível perceber que depois de um ponto de máxima, a fração empírica de atualizações diminui com a densidade do grafo.

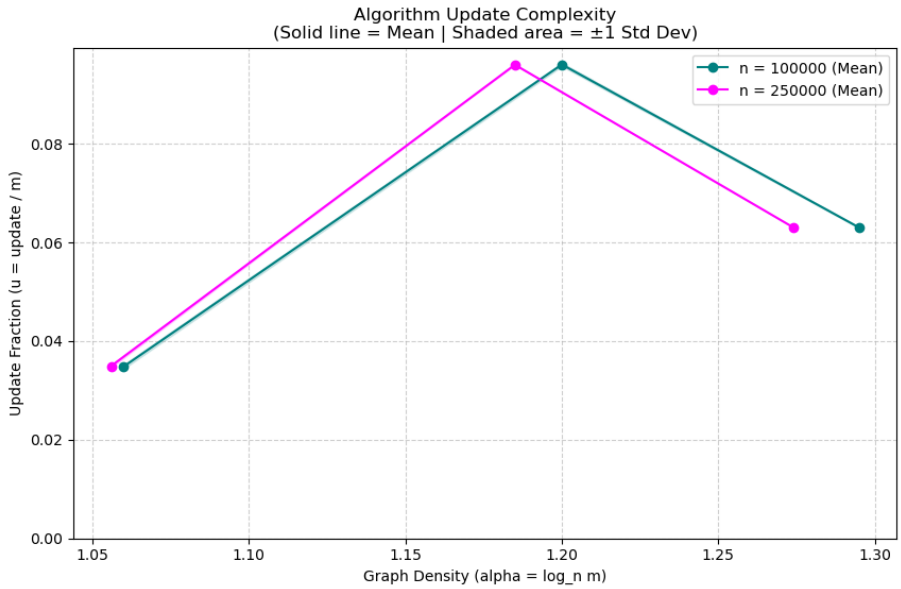

Figura 1.4 – Fração de atualizações u em relação à densidade do grafo α.

1.4.4 Análise do Caso Médio: Valor Esperado de Noshita

A proposição de Noshita diz que o número de update gira em torno de $n \ln(m/n)$ incidências, para arestas com valores aleatórios.

A curva empírica da Fig. 1.5 mostra a associação do valor resultado dos testes com o limite modelado. O número de *Updates* cresce com o grau k do Heap, que resulta em árvores mais largas.

1.4.5 Avaliação do Escalonamento Assintótico

Análise com o objetivo de verificar o escalonamento da complexidade temporal teórica $O(m \log_k n + n \cdot k \log_k n)$. Para isso o tempo de execução medido pela fórmula teórica $T/((n+m)\log n)$ foi normalizado. Nas Figuras 1.6 e 1.7, as linhas plotadas são praticamente horizontais, mesmo com o crescimento exponencial do tamanho do grafo. A implementação escala de acordo com a complexidade teórica.

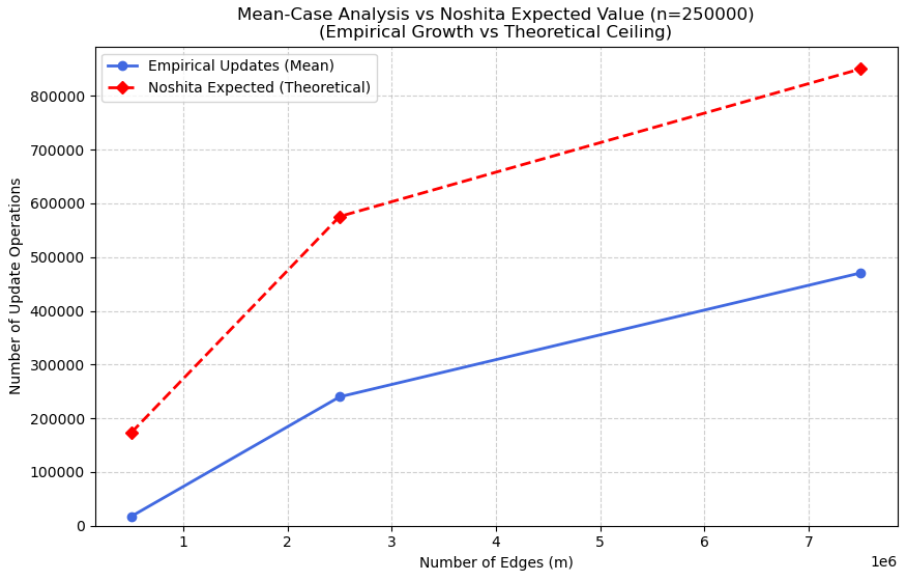

Figura 1.5 – Análise de caso médio das atualizações vs. Valor Esperado de Noshita.

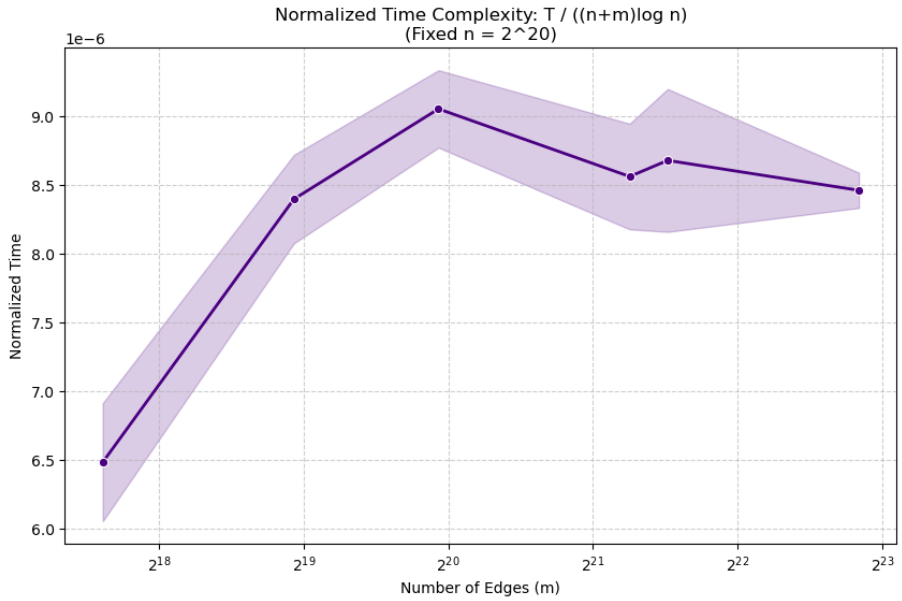

Figura 1.6 – Complexidade de tempo normalizada variando o número de arestas (m).

1.4.6 Análise de Regressão Linear Múltipla

Aplicando regressão linear com a equação $T(n,m) \approx a \cdot n^b \cdot m^c$, compara-se o tempo real de execução com o tempo previsto pelo modelo da Fig. 1.8.

1.4.7 Desempenho no Mundo Real (Desafio DIMACS)

A etapa final utilizou o grafo de estradas USA.gr, que tem quase 24 milhões de vértices. Com a análise de tempo (Figura 1.9) é possível perceber que o gargalo de

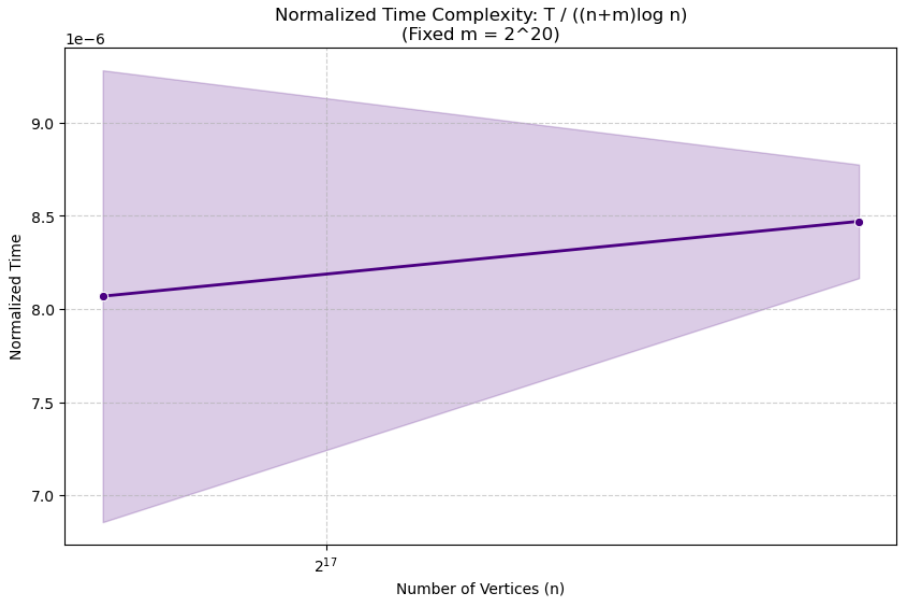

Figura 1.7 – Complexidade de tempo normalizada variando o número de vértices (n).

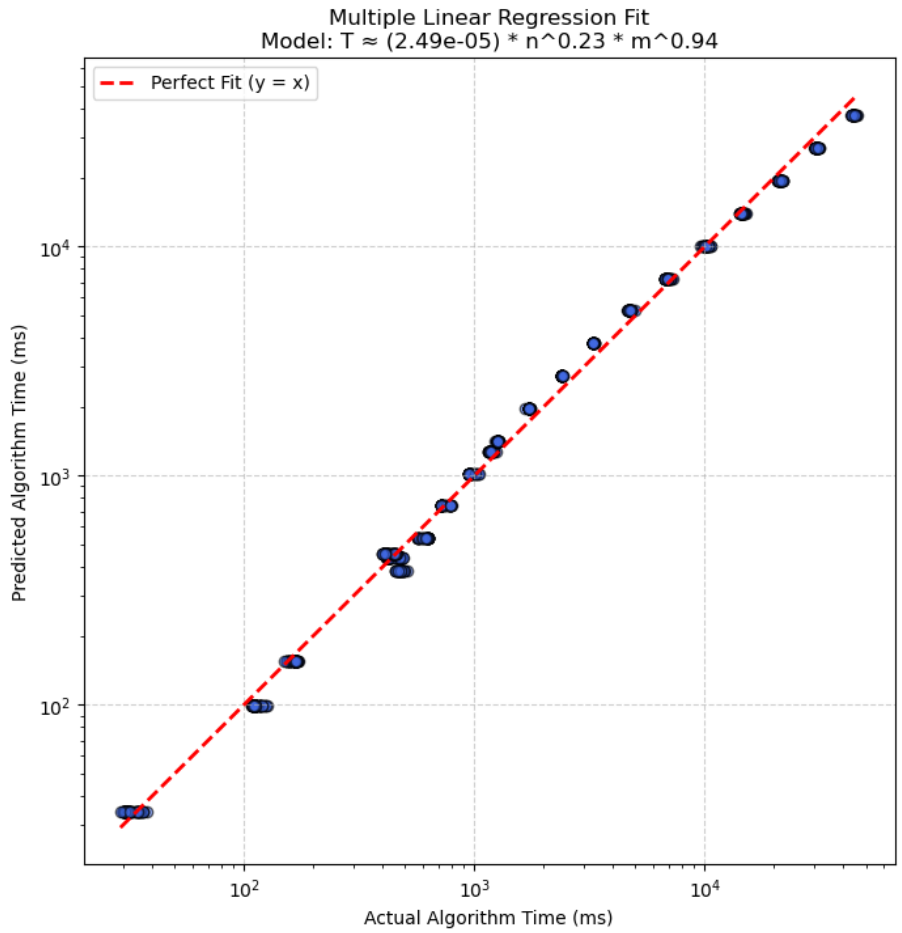

Figura 1.8 – Gráfico de dispersão Log-Log: Tempo Real vs. Previsto pelo Modelo.

desempenho está na operação de E/S (leitura do arquivo do disco para a memória).

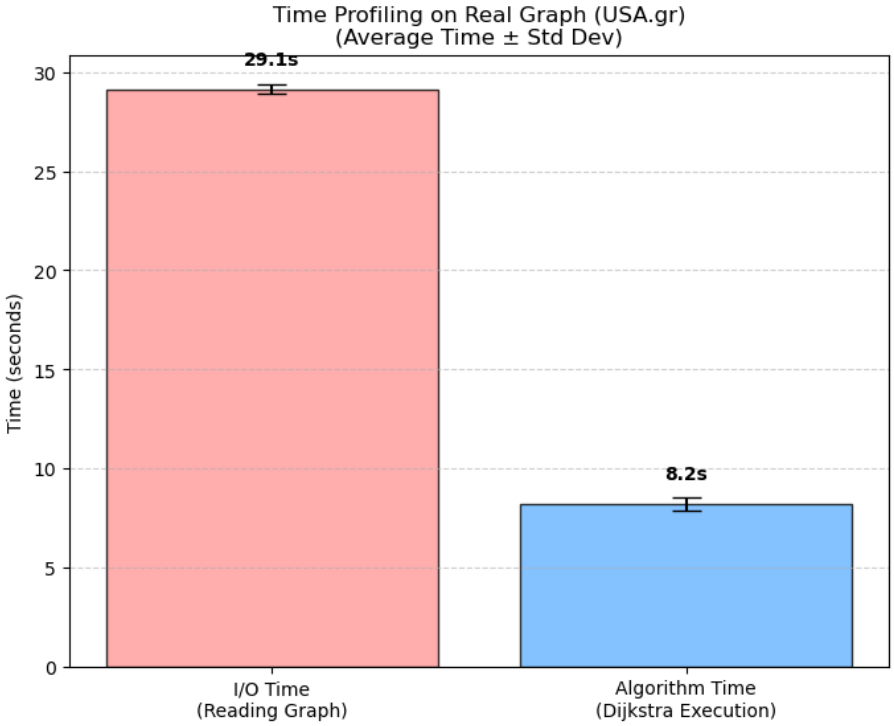

Figura 1.9 – Perfil de tempo evidenciando o gargalo de E/S de disco.

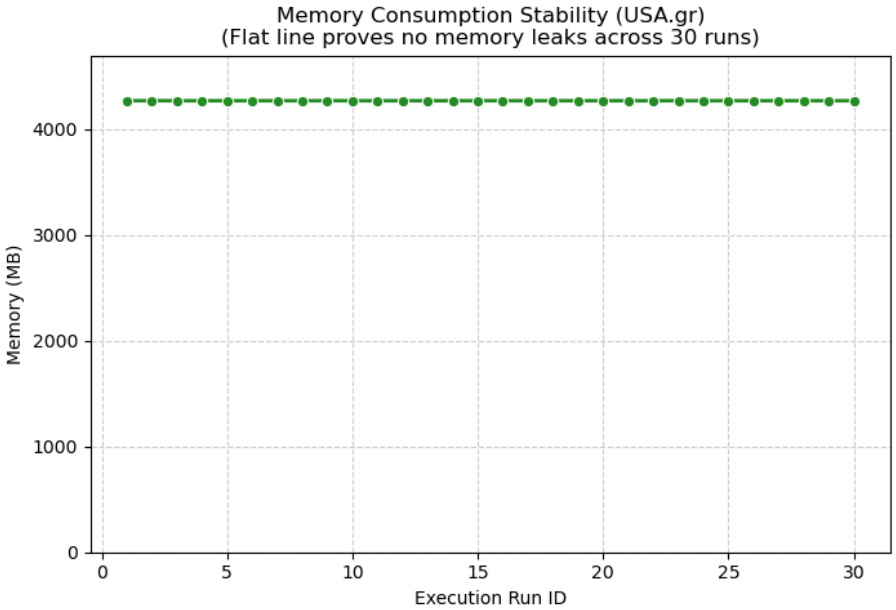

Figura 1.10 – Consumo e estabilidade de RAM ao longo das execuções (Grafo USA).

1.5 Conclusão

A implementação respeitou os limites téoricos de complexidade e o modelo de caso médio. O algoritmo escalou corretamente para processar o grafo USA.gr.

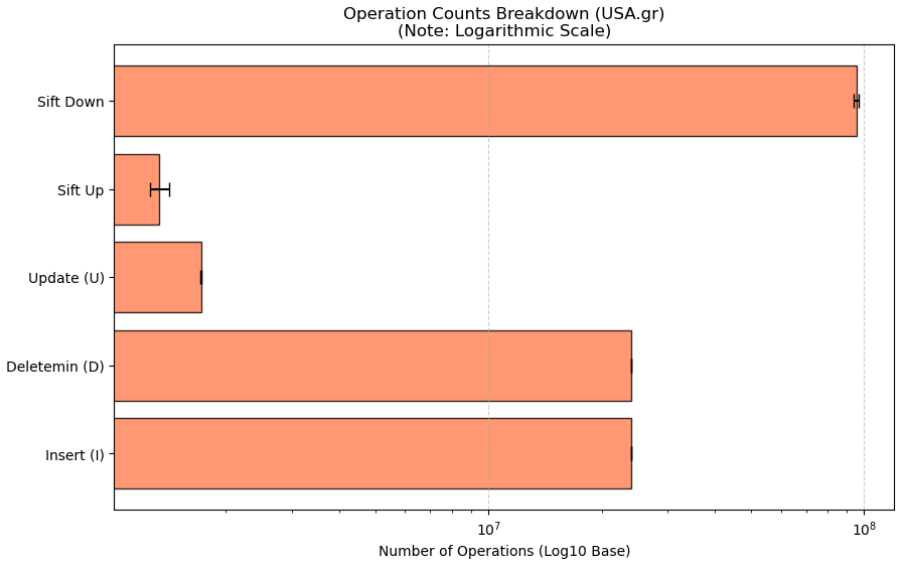

Figura 1.11 – Detalhamento de operações executadas em rede rodoviária real.